

The 8085 Master Manual

Volume II: Inside the Silicon
(The Internal Anatomy & Machine Logic)

A Grassroots Engineering Guide

Moving from external pins to internal computation.

"A machine is not intelligent; it is merely obedient to the laws of logic."

Chapter 1: Crossing the Silicon Boundary

1.1 The Internal Bus Architecture

In Volume I, we mastered the external system bus—the 16-bit Address Bus and the 8-bit Data Bus that connect the microprocessor to the motherboard. Now, we must follow the electrical signals as they cross the physical pins and enter the silicon die of the 8085.

Inside the processor, space is incredibly limited. The engineers could not afford to route dozens of independent 8-bit wires between every single internal component (ALU, Registers, Instruction Decoder). Instead, the 8085 utilizes an **8-bit Internal Data Bus**.

Think of the internal data bus as a central highway running down the middle of the chip. Every major internal component is connected to this single highway. Because it is a shared resource, only *one* component can place data on the bus at any given microsecond, and only *one* component can read from it. This strict orchestration is managed by the Timing and Control Unit.

1.2 The Arithmetic Logic Unit (ALU)

The absolute heart of the microprocessor is the Arithmetic Logic Unit (ALU). The ALU is a massive array of combinational logic gates (AND, OR, XOR, NOT, and Full Adders). It has no memory of its own; it is a purely reactionary circuit.

THE MATHEMATICAL BOUNDARY OF THE 8085

The 8085 is classified as an *8-bit microprocessor*. This classification is derived directly from the width of its ALU. The ALU can only process binary numbers that are exactly 8 bits wide (from **00000000** to **11111111**). Any mathematical operation exceeding the value of 255 (Decimal) must be broken down into multiple 8-bit steps by the software.

Because it is a combinational circuit, the ALU requires its inputs to be held perfectly stable during the calculation. To achieve this, the ALU is sandwiched between two dedicated 8-bit holding registers: the **Accumulator** and the **Temporary Register**.

Chapter 2: The Core Registers (A and Temp)

2.1 The Accumulator (Register A)

The Accumulator is the most frequently utilized register in the entire 8085 architecture. It is an 8-bit register that serves a dual purpose:

1. **Pre-Calculation:** It holds one of the operands (inputs) before an ALU operation begins.
2. **Post-Calculation:** It is the inescapable destination for the result of *every* arithmetic and logical operation performed by the ALU.

If you execute an instruction to add Register B to the Accumulator (**ADD B**), the ALU takes the value from the Accumulator, takes the value from Register B, adds them together, and immediately overwrites the old value in the Accumulator with the new sum.

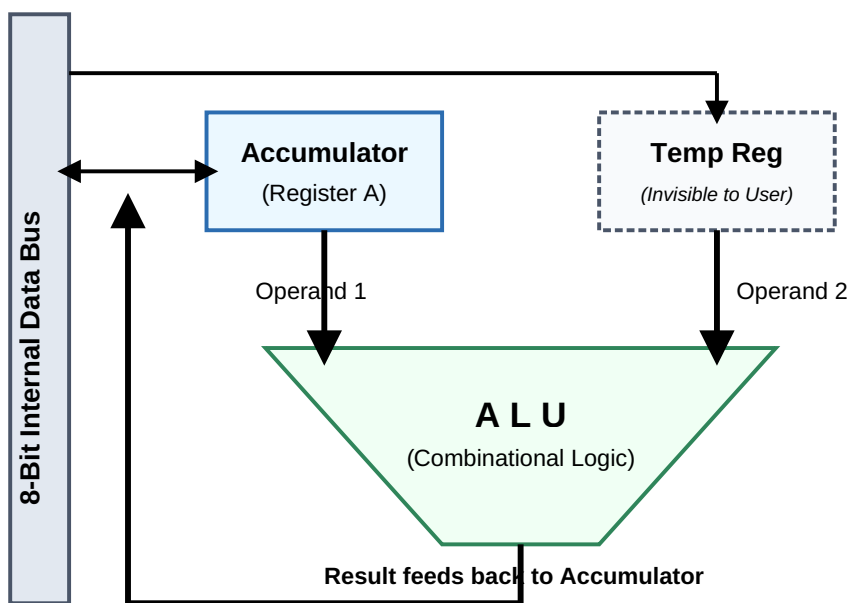
2.2 The Invisible "Temporary Register"

We stated that the ALU needs two inputs. One comes from the Accumulator. Where does the second come from?

If we issue the command **ADD B**, the data from Register B must travel along the Internal Data Bus to reach the ALU. However, the Internal Data Bus might be

needed for other tasks during the same machine cycle. To free up the bus, the 8085 has a hidden, internal 8-bit **Temporary Register (W/Z)** physically attached to the second input of the ALU.

When **ADD B** is executed, the CPU silently copies the contents of Register B into the Temporary Register. The Temporary Register holds the data perfectly steady for the ALU, allowing the ALU to process the addition while the internal bus goes to sleep or prepares for the next task. The programmer can never access this Temporary Register directly; it belongs entirely to the hardware.



Chapter 3: The Dashboard of Truth (Flag Register)

3.1 The Necessity of Flags

When the ALU adds two numbers, it pushes the 8-bit result into the Accumulator. But what if the numbers were too large and the addition caused an overflow? What if the result was exactly zero? The Accumulator only holds the 8 bits of data; it cannot store meta-information *about* the data.

To solve this, the 8085 includes a dedicated 8-bit register called the **Flag Register**. However, only 5 of its bits are actually connected to flip-flops. These flip-flops act as a dashboard of warning lights. They monitor the ALU's output and automatically turn on (1) or off (0) based on the mathematical properties of the result.

8085 Flag Register Bit Format

S	Z	X	AC	X	P	X	CY
D7	D6	D5	D4	D3	D2	D1	D0

3.2 Dissecting the Five Flags

1. Sign Flag (S) - Bit D7

The 8085 can perform "Signed Arithmetic" using two's complement representation. In this system, the Most Significant Bit (MSB, which is D7 of the accumulator) dictates the sign of the number. If D7 is 1, the number is considered negative. If D7 is 0, it is positive. The Sign Flag is literally a direct copy of the Accumulator's D7 bit after an operation.

2. Zero Flag (Z) - Bit D6

This is arguably the most important flag for program logic (loops and branching). The Zero Flag is set to **1** if, and only if, the entire 8-bit result in the Accumulator is exactly **0000 0000**. If even a single bit is 1, the Zero flag is reset to **0**. Internally, this is achieved by feeding all 8 bits of the accumulator through a massive NOR gate.

3. Auxiliary Carry Flag (AC) - Bit D4

This flag is used exclusively for Binary Coded Decimal (BCD) arithmetic. An 8-bit byte is composed of two 4-bit "nibbles". The AC flag is set to **1** if an addition generates a carry from the lower nibble (D3) into the upper nibble (D4). *Note: The programmer cannot test the AC flag directly using a jump instruction; it is used internally by the DAA instruction.*

4. Parity Flag (P) - Bit D2

Parity refers to the number of '1's in the binary result. If the result has an **Even** number of 1s (e.g., **0101 0011** has four 1s), the Parity Flag is set to **1**. If the result has an **Odd** number of 1s, the flag is reset to **0**. This is often used for detecting errors in serial communication.

5. Carry Flag (CY) - Bit D0

If an addition of two 8-bit numbers exceeds 255 (**1111 1111**), the 9th bit "spills over". This overflow is caught by the Carry Flag. It is set to **1** if a

carry is generated out of the MSB (D7). In subtraction, it acts as a "Borrow" flag.

3.3 Mathematical Derivation: Tracing the Flags

To truly understand the MH methodology, we must trace a mathematical operation at the binary level and observe the hardware's reaction.

EXECUTION TRACE: ADI 92H

Assume the Accumulator initially holds the value 7FH. We execute the instruction ADI 92H (Add Immediate). The ALU must add 7FH and 92H.

STEP 1: BINARY CONVERSION

7FH = 0111 1111

92H = 1001 0010

STEP 2: THE ALU FULL ADDER CIRCUIT EXECUTION

```

(Carry Row)  1 1 1 1  0 0 1 0
              0 1 1 1  1 1 1 1  (7FH)
            + 1 0 0 1  0 0 1 0  (92H)
            -----
Final Carry: 1 0 0 0 1  0 0 0 1  (Result: 11H)

```

STEP 3: EVALUATING THE FLAG FLIP-FLOPS

- **ACCUMULATOR RESULT:**

0001 0001 (Hex 11H).

- **SIGN (S):**

Look at D7 of the result. It is 0. Therefore,

S = 0

.

- **ZERO (Z):**

The result is 11H, not 00H. Therefore,

Z = 0

.

- **AUX CARRY (AC):**

Look at the transition from D3 to D4. The lower nibble addition was $1111 + 0010 = 1\ 0001$. A carry of 1 crossed the nibble boundary.

Therefore,

AC = 1

.

- **PARITY (P):**

Count the 1s in the result $0001\ 0001$. There are exactly two 1s. Two is an even number. Therefore,

P = 1

.

- **CARRY (CY):**

Look at the Final Carry generated out of D7. It is 1. Therefore,

CY = 1

.

FINAL FLAG REGISTER STATE:

0001 0101 (Hex 15H).

This deterministic, mechanical evaluation of bits is exactly how the silicon die operates. The processor does not "know" math; it simply propagates electrical states through logical gates.

Chapter 4: The Scratchpad (General Purpose Registers)

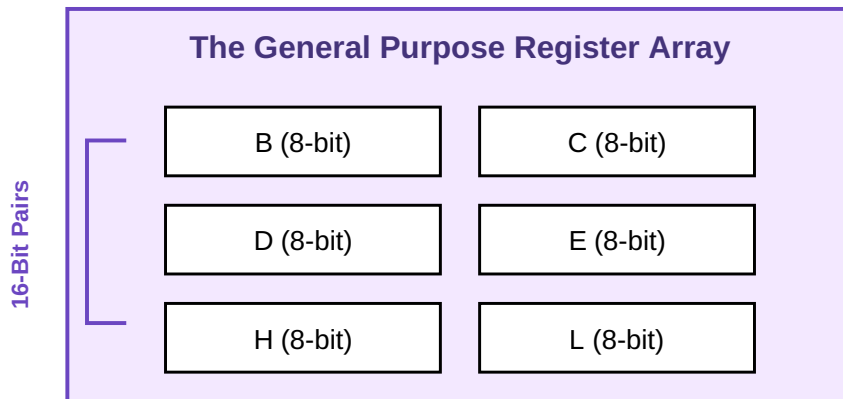
4.1 The Need for Internal Storage

Fetching data from the external Memory (RAM) takes a minimum of 3 clock cycles (T-states). If a complex algorithm requires manipulating the same piece of data repeatedly, fetching it from RAM every time would bottleneck the processor.

To solve this, the 8085 provides six 8-bit **General Purpose Registers** located physically inside the CPU, directly adjacent to the Internal Data Bus. Moving data between these registers takes only 1 T-state (and is usually hidden within the Opcode Fetch cycle itself).

4.2 The Register Array: B, C, D, E, H, L

These six registers are electrically identical 8-bit D-flip-flop arrays. The programmer can use them to store any 8-bit data.



4.3 Register Pairing and the 'M' Pointer

The Address Bus of the 8085 is 16 bits wide. Therefore, memory addresses are 16 bits long. An 8-bit register cannot hold a memory address.

To solve this, the architecture allows the registers to be logically concatenated into **16-bit Register Pairs: BC, DE, and HL**. When used as a pair, the first register (e.g., B) holds the high-order byte of the 16-bit number, and the second register (C) holds the low-order byte.

The Special Role of HL: While BC and DE can hold 16-bit data, the **HL Pair** is physically wired to the Address Buffer. It acts as the default Memory Pointer. In assembly language, the operand **M** (Memory) explicitly refers to "the memory location whose 16-bit address is currently stored in the HL register pair."

If HL contains **2050H**, the instruction **MOV A, M** commands the CPU to place **2050H** on the Address Bus, execute a Memory Read cycle, and pull the resulting data byte into the Accumulator.

Chapter 5: The Sequencers

Combinational logic determines *what* happens. Sequential logic determines *when* it happens. The 8085 relies on two highly specialized 16-bit registers to navigate the vast ocean of external memory: the Program Counter and the Stack Pointer.

5.1 The Program Counter (PC)

The microprocessor is inherently "dumb". It does not know the purpose of the program it is running. It only knows how to do one thing: fetch the next instruction and execute it.

The **Program Counter (PC)** is a 16-bit register that dictates this flow. Its sole responsibility is to hold the memory address of the *next* byte to be fetched.

THE AUTO-INCREMENT ARCHITECTURE

The PC is physically attached to an internal 16-bit Incrementer/Decrementer circuit.

During the T₁ state of an Opcode Fetch cycle, the contents of the PC are dumped into the Address Buffer and sent out to the motherboard. *Immediately* after this, the internal Incrementer adds +1 to the PC. By the time the data arrives in T₃, the PC is already pointing to the subsequent memory address, ready for the next cycle.

When the programmer writes a jump instruction (e.g., **JMP 4000H**), the processor fetches the instruction, and then the hardware forcibly overwrites the contents of the PC with **4000H**. On the next machine cycle, the processor naturally places **4000H** on the address bus, successfully "jumping" to the new location.

5.2 The Stack Pointer (SP)

The **Stack Pointer (SP)** is a 16-bit register that maintains a specialized data structure in external RAM known as the **Stack**. The Stack operates on a Last-In-First-Out (LIFO) principle.

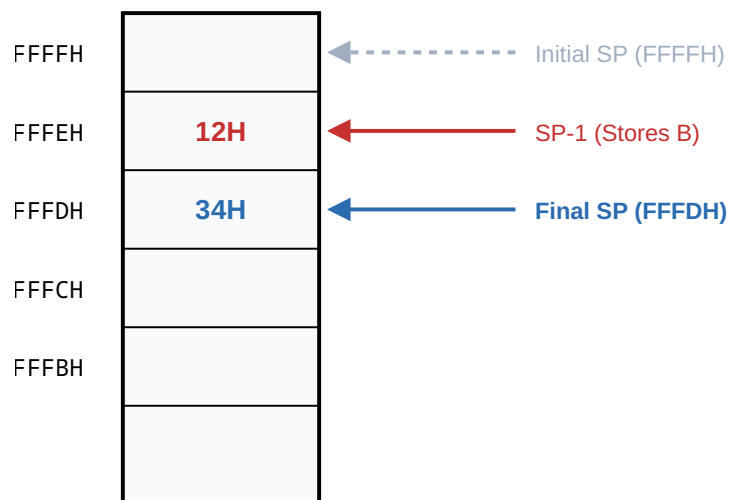
The Stack is not inside the CPU; it is an area of the external RAM. The SP merely holds the 16-bit address of the "Top" of this stack. By convention in the 8085, the Stack grows backwards (from high memory addresses down to lower ones).

The Mechanics of PUSH and POP

When the CPU needs to save a 16-bit register pair (like BC) temporarily, it uses the **PUSH B** instruction:

1. The SP is decremented by 1 ($SP = SP - 1$).
2. The high-order byte (Register B) is written to the memory location pointed to by the new SP.
3. The SP is decremented by 1 again ($SP = SP - 1$).
4. The low-order byte (Register C) is written to the new memory location.

Stack Memory Map during PUSH B (Assume B=12H, C=34H)



The **POP** instruction reverses this process: it reads the low byte, increments the SP, reads the high byte, and increments the SP again. This LIFO mechanism is essential for handling nested Subroutines and hardware Interrupts, ensuring the CPU can always find its way back to its original execution state.

Chapter 6: The Translator

We have data, we have registers, and we have an ALU. But how does a simple 8-bit binary number (fetched from memory) suddenly cause the CPU to route data from Register C, send it to the ALU, and store the result in the Accumulator?

6.1 The Instruction Register (IR) and Decoder

During the T₃ state of the Opcode Fetch cycle, the 8-bit instruction code (the Opcode) arrives on the Internal Data Bus. The CPU cannot put this in the Accumulator or a general register, because it is not data to be manipulated; it is a command to be executed.

The opcode is routed exclusively into the **Instruction Register (IR)**. This register is not accessible to the programmer. Its sole job is to hold the opcode stable.

Connected directly to the output of the IR is the **Instruction Decoder**. This is a massive web of combinational logic gates. It takes the 8-bit input (which has 256 possible unique patterns) and decodes it.

If the pattern is **0100 0111** (which is **MOV B, A**), the Decoder activates a specific output wire.

6.2 The Timing and Control Unit

The single activated wire from the Decoder feeds directly into the **Timing and Control Unit**. This unit is a complex state machine driven by the system clock.

Upon receiving the decoded signal for **MOV B, A**, the Timing and Control Unit generates a precise sequence of microscopic electrical pulses across the internal control bus:

1. Pulse 1: Assert the "Output Enable" pin of the Accumulator, dumping its contents onto the Internal Data Bus.
2. Pulse 2: Assert the "Load Enable" pin of Register B, forcing it to capture whatever is currently on the Internal Data Bus.

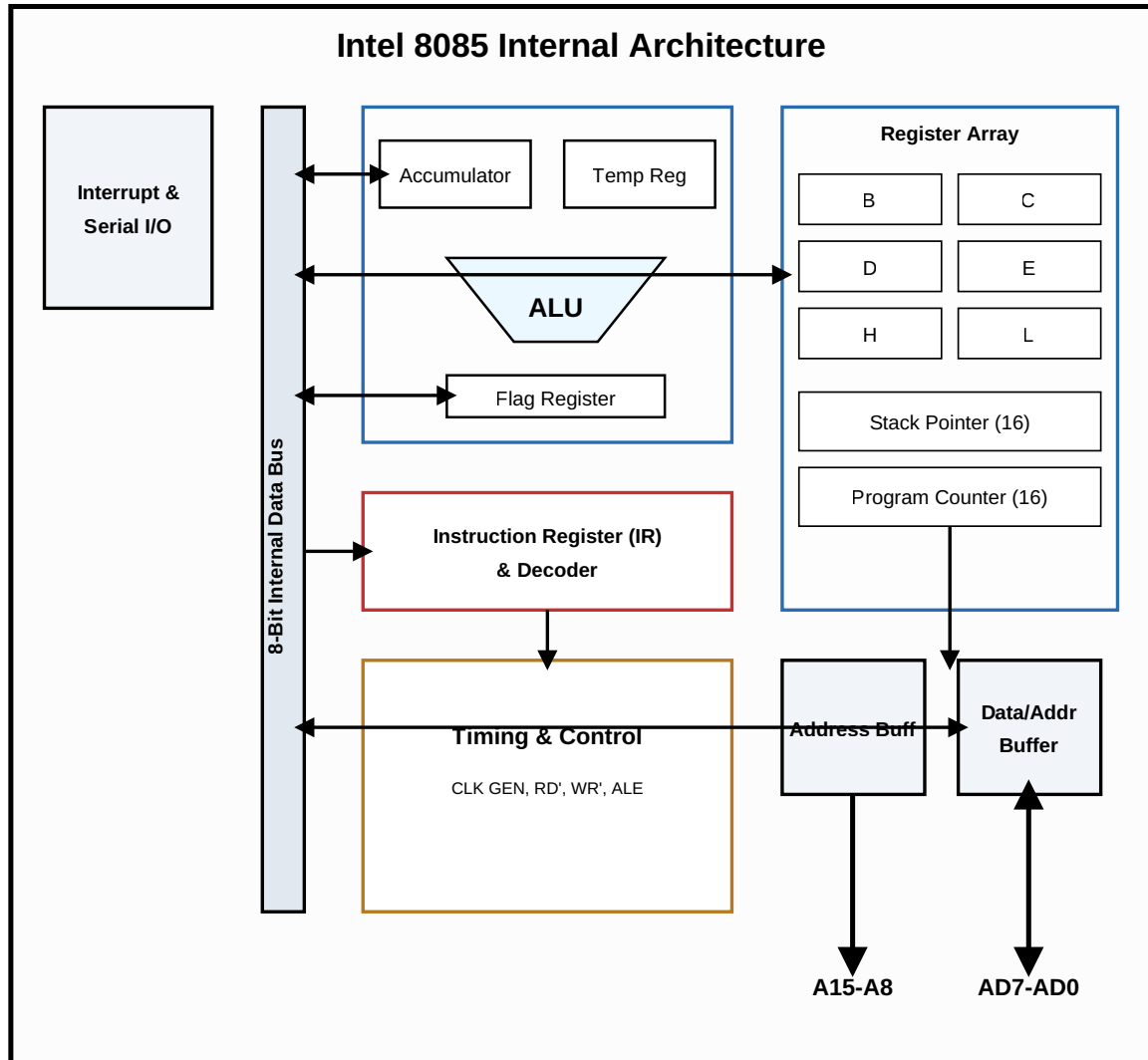
This entire sequence occurs during the **T₄** state (the Bus Idle state). Once completed, the instruction is officially "Executed," and the Timing Unit signals the Program Counter to initiate the next Opcode Fetch cycle.

Chapter 7: Synthesis (The Complete Architecture)

We have now traced the logical derivation of every major internal component of the Intel 8085 microprocessor.

- We built the **Internal Data Bus** to route signals.
- We built the **ALU, Accumulator, and Temp Register** to execute math.
- We built the **Flag Register** to monitor the results.
- We built the **General Purpose Registers (B-L)** for fast scratchpad memory.
- We built the **Program Counter and Stack Pointer** to sequence the architecture.
- We built the **Instruction Decoder and Timing Unit** to orchestrate the chaos.

When we assemble all of these components onto a single die of silicon, we achieve the complete Internal Architecture Block Diagram of the 8085.



End of Volume II